

Imager: a domain-specific language and smart microscopy platform for advanced fluorescence imaging

Sergey Abakumov, Gerard Carrera i Cardona, Robin van den Eynde,
Tom Schrijvers, Peter Dedecker

Department of Chemistry and Department of Computer Science, KU Leuven, Belgium

peter.dedecker@kuleuven.be

June 18, 2026

Abstract

Fluorescence imaging is an essential method though considerable challenges emerge in executing and preserving complex experiments. We present Imager, a microscope control solution based on the use of a domain specific language (DSL). This DSL is specifically designed to describe a broad range of fluorescence microscopy experiments in a way that is intuitively clear, allowing even non-expert users to straightforwardly and safely describe complex experimental pipelines. The resulting programs are then executed by the microscope directly. By building on the intuitive, clear DSL, our approach dramatically lower the technical expertise required to specify complex measurement programs and greatly facilitates FAIR (findable, accessible, interoperable, and reusable) data use and preservation. We also show that our design readily allows clear and tractable ‘smart’ (data-guided) microscopy. Our DSL-based design is intrinsically compatible with AI-assisted measurement optimization, supporting collaborative interactions based on the shared context and knowledge embedded in the DSL. Overall the Imager design sets new standard for accessible, FAIR and smart microscopy, both for simple and complicated instruments and experimental designs.

This is a pre-preprint describing our Imager domain-specific language and software to control optical microscopes. The figures are largely placeholders and we sorely lack references to the current state of the art. Our apologies to anyone affected! However, the current document does give an overview of the technology and the underlying reasoning motivating our work.

Complex instruments require sophisticated control software, particularly when used in dynamic and demanding settings such as research. At a minimum, it must provide accessible, robust, and safe operation suitable for extended and routine use. Furthermore, it should permit highly flexible applications to provide room for development and to meet new challenges over the service life of the microscope.

The requirements for ease of use, reliability, and flexibility can be difficult to reconcile. Most software packages address this by providing a limited set of experiment types that can be configured using an intuitive interface, combined with a scripting or programming environment that provides low-level access using a more general-purpose programming language such as Python. This scripting often requires strong technical expertise because it needs both a strong understanding of programming as well as a strong understanding of the experimental hardware. Furthermore, the resulting programs can be difficult to understand and incorporate specific knowledge of the environment in which they are operating. This information not only strong limits the set of users that can successfully achieve this, but also severely challenges FAIR data use and data preservation via the low operating level required as well as the many (proprietary) systems and formats in use.

Recent years have seen the emergence of ‘smart microscopy’, imaging experiments in which the control software takes autonomous, data-driven decisions so that the experimental hypotheses can be better addressed. This means that even more flexibility needs to be incorporated in the imaging, requiring logic that directly interacts with the flow of information, decision taking, and hardware execution. The resulting complexity typically requires expert knowledge to understand, becomes even more dependent on the particulars of the operating environment, and poses even larger challenges for data preservation and sharing. Basic questions such as which ‘smart’ decisions are taken, the logic that is taken to make these, and how this affects the measurements become difficult to assess without detailed knowledge of the code involved, which may rely on environments or tools that may be associated with vendor lock-in or may disappear or change at any time.

Our research group has a long track record in the development of innovative imaging modalities that require considerable flexibility to implement. Over the past decade, we have therefore pursued an alternative instrument control strategy that can mitigate or even eliminate many of these concerns. We did so by adopting an innovative strategy, in which we approach this challenge not as a software engineering but rather as a *language* challenge. Fundamentally, a microscope user has some idea of what they want their experiment to do. The microscope also ‘knows’ what it can do. The challenge is enabling the user to communicate this intent to the microscope so that both agree on what is to be done. Expressing ideas or communicating intent is exactly why languages exist, so it makes sense to focus at this level.

We therefore set out to define a language designed to describe a broad variety of fluorescence microscopy experiments in a way that is intuitively clear to any user with a minimal imaging proficiency. The overall design of our ‘Imager’ language is shown in Figure 1. Briefly, our language consists of a number of primitives that can be combined into arbitrarily complex programs. These primitives consist of ‘actions’ that execute particular experiment steps and ‘flow control’ elements that sequence or order the actions. The resulting programs take the form of trees.

Despite its apparent simplicity, the Imager language makes it possible to express a broad range of measurement programs. Furthermore, its tree-based nature means that the programs can be created in an entirely graphical manner, involving the simple dragging and rearranging of the program elements. Users with no programming experience can rapidly design complex programs. Furthermore, it is impossible to create invalid programs, in that any program that can be constructed can also be executed successfully. Overall, complex measurements can be defined in a user-friendly manner that requires essentially zero programming experience from the end user, with a clarity that reduces the possibilities of mistakes or errors, by ensuring that the resulting programs are well-formed and that they are expressed in a consistent and simple manner. More fundamentally, our language is an example of a domain-specific language (DSL), meaning that it incorporates domain-specific knowledge to deliver greater clarity and ease of use.

Within our model, the microscope acts as an interpreter for programs written in the Imager language: once the user has defined a measurement using the DSL, the resulting program is transmitted to the microscope to conduct the measurement. The microscope control software then simply ‘walks’ the program tree in a depth-first manner, performing the actions associated with each primitive and continuing until the entire program has been processed.

Our language-focused approach has three major benefits: first, we can design the language to be simple, clear, and consistent, dramatically lowering the barrier for the users. Secondly, by its very definition, the language completely describes the experiment, meaning that the full program specification is sufficient to understand and share the experiment, even across different platforms. Our solution is therefore vendor- or environment agnostic. It does not matter how the software using the DSL is implemented, as long as it correctly interprets the language. Furthermore, because the DSL is both clear and environment-agnostic, we immediately solve a long-standing barrier that has hindered FAIR data usage, since our measurements are described in a clear way with unambiguous meaning, even if knowledge on the operating environment were to be lost entirely.

We created a software implementation of our DSL and imaging concept. The overall architecture of the system is depicted in Figure **fig:ProgramElements**. Briefly, we divided the software into two components: a microscope controller that interacts with the hardware (Imager.Core) and a graphical

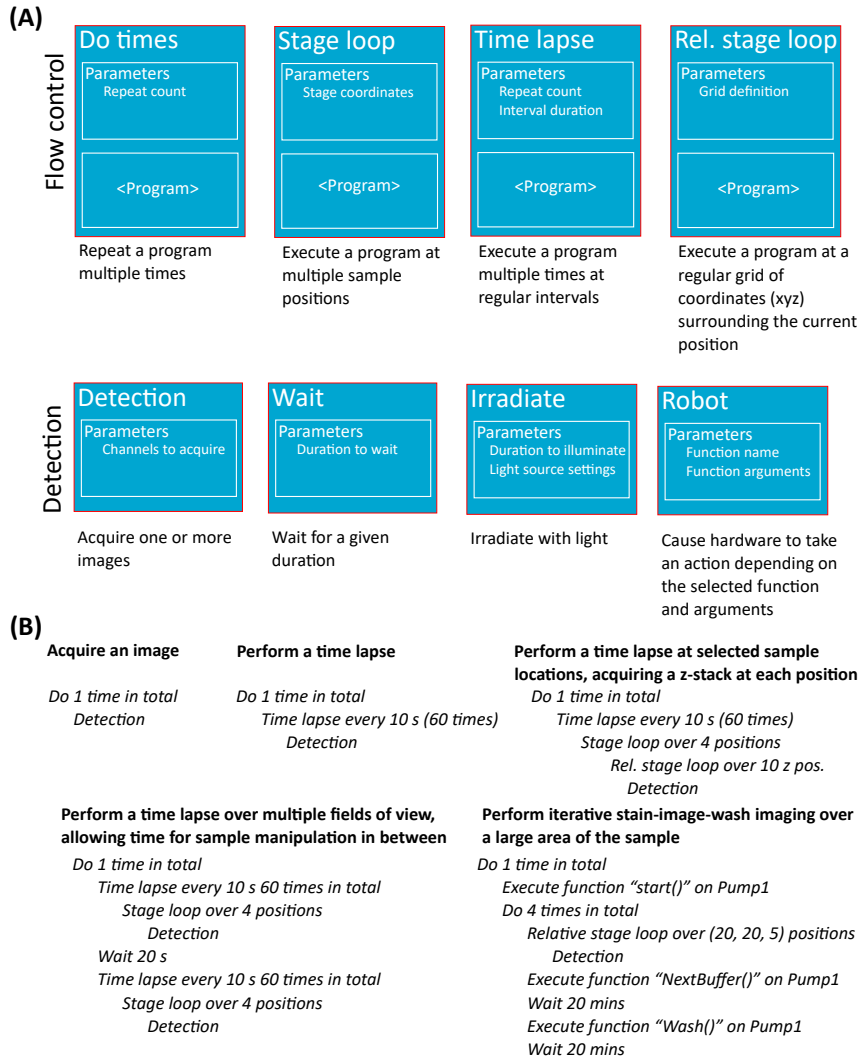


Figure 1: (A) Graphical illustration of the different primitives part of the DSL. (B) Example programs illustrating the wide range of experiments that can be implemented. Increasing indentation signifies that the primitive is a child of the element immediately preceding it.

front-end that provides the interaction with the user (Imager.GUI). The controller is responsible for the data acquisition, while the front-end allows the user to graphically set up the measurement, start the acquisition, and view and save the acquired data. We chose to implement these as two separate programs that communicate via a client-server architecture, in which the controller acts as the server. Our choice to implement these as two separate programs has three advantages: (i) it clearly separates the software into two separate elements that communicate mainly via the DSL, enforcing our core design, (ii) it permits the use of different programming languages and environments for setting up the two components, and (iii) it makes it possible to swap out different front- and back-ends as long as they agree on the shared DSL.

The implementation of the microscope control component was guided by the need to deliver very high performance and reliability. Controlling a complex microscope is a strong challenge due to the technical details of the hardware communication as well as the need for precise synchronization and timing as well as the acquisition and transfer of large amounts of data. For these reasons, we chose to implement this system in Haskell, a purely functional programming language known for its strong type system and for its data immutability. While its programming style can be challenging to adopt for programmers familiar with more mainstream programming languages, Haskell offers strong benefits including high reliability, ease of refactoring, straightforward concurrency and error handling, and good runtime performance. The resulting software has proven extremely robust, easily acquiring terabytes of data in a single experiment if required, as well as handling multiple cameras at once at data rates easily exceeding several gigabytes per second.

Controlling the experimental hardware is done by having the Haskell runtime leverage a plugin-based system, in which the hardware communication is handled by plugins written in the C++ language. These plugins categorize the associated equipment into a number of different types, making it possible to associate these with a meaning within the DSL. A particular equipment, for example, could be declared to be a camera, a filter wheel, or a light source. We also created a plugin that interacts with the Micro-Manager runtime, in principle offering access to hardware accessible via this system.

The original GUI frontend for Imager was written in the Igor Pro environment. A second version was written in the C# programming language, exposing a user-friendly interface to the overall measurement process. The acquired data is stored in a TIFF file containing OME-XML metadata, including a full description of the measurement program that was used. By design therefore, the saved data contains all the details necessary to reproduce it.

The overall strategy underlying our Imager approach has shown its value over a broad range of applications, having been in use in the hosting research group for over 10 years, during which time it has acquired a large amount of data that has been used in a variety of publications. Some example in include Refs. [1–11]). Its generic design has also allowed it to be used for spectroscopic measurements that required careful light control but that did not involve microscopy [6, 12].

We also wanted to augment our approach with the ability to perform smart microscopy, while still retaining the overall clarity and reliability inherent to our vision. To achieve this, the controller must be capable of dynamically updating the measurement program while it is executing. However, we wanted to ensure that the information flow between the imaging and the smart functionality was clearly specified as part of the DSL, and that it would be clear just what aspects of the measurement program are affected. Furthermore, we wanted to ensure that the resulting programs were still archivable and retained their FAIR properties.

To achieve this, we expanded our DSL by introducing the concept of ‘smart programs’, self-contained units of code that accept one or more acquired images as input and that provide decisions that affect the flow of the measurement. To clarify the information flow and make it easier to reason about such a measurement, we introduced clear structuring on how such smart programs can be introduced: Every smart program is explicitly coupled to one or more detection events in the measurement program, meaning that it will receive those images as they are acquired. Likewise, every smart program must have an output that is connected to one of the primitives in the program. For example, connecting the output of a smart program to a ‘do times’ primitive means that the smart program provides the number of times this primitive will execute the program elements it contains.

Likewise, coupling its output to a ‘stage loop’ means that the smart program provides the positions at which this primitive executes, and so on for the other measurement elements. The resulting strategy not only clarifies the information and decision flow, but also ensures clarity when assessing which decisions are taken and how they affect the measurement.

To allow maximum flexibility, we decided to adopt Python as the language for smart programs, meaning that these are capable of essentially unlimited functionality. Upon starting a measurement, the experimental program consists not only of the measurement description, expressed using our DSL, but also contains the full source code of any smart programs as well as the connectivity information associated with these. In this way we ensure that the measurement description contains all of the information required to fully run and reproduce the experiment, and that all of these components are stored with the output data. As part of executing the measurement, the measurement controller starts up a separate Python process for every smart program, including the provision of a wrapper that simplifies data transfer and decision return. The controller also logs every decision made by a smart program, allowing the full execution history of the program to be stored for preservation and later analysis.

The use of a general-purpose programming language such as Python offers maximum flexibility and aligns with the popularity of this language in data analysis and other scientific applications, though it also raises the technical expertise required for successful usage. We reasoned that this trade-off was acceptable in view of the flexibility it permits. Furthermore, the complexity of the smart programs is dramatically reduced because these do not need to be aware of the measurement or microscope details, which are all handled by the controller. All they have to do is to accept images provided to it, including their metadata, and the generate decisions in whichever way is best. Fundamentally, this scheme is also implementation agnostic, in the sense that any functionality that accepts images and returns decisions can be used.

Figure 2 shows a few examples of such smart programs in action, including the tracking of live objects as well as a more toy example in which the microscope is asked to solve a maze.

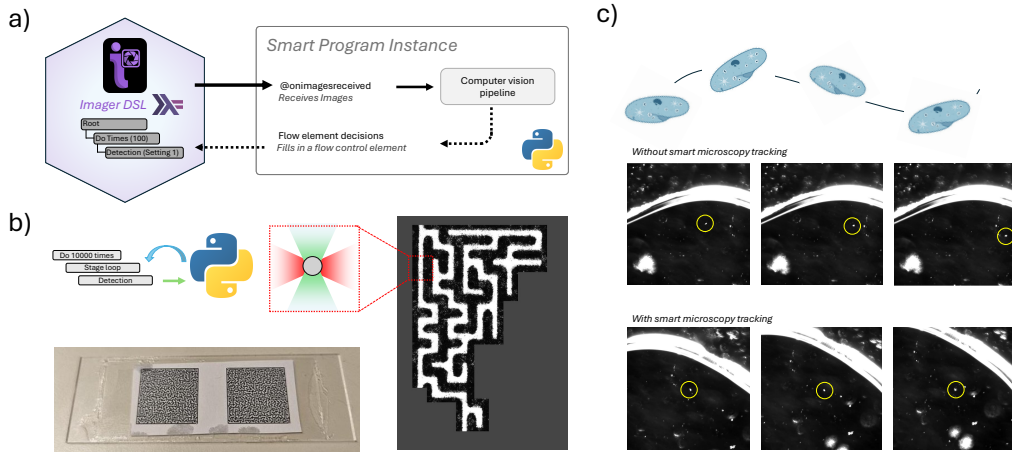


Figure 2: a) Overview of the smart microscopy subsystem and its integration within the DSL workflow. b) Maze traversal experiment showing the stitched field of view alongside the corresponding control program. c) Live tracking of micro-organisms enabled by the smart microscopy subsystem.

Finally, our approach also offers advantages when combined with the use of generative artificial intelligence (AI). The use of AI will strongly increase the power of microscopy by assisting non-expert users in setting up or optimizing their microscope workflow, or more generally in assisting any user in precisely designing their experiment to best address their hypotheses. However, a key question is how this assistance is to be provided. One possibility is for the AI to directly generate source code (e.g. written in Python) that will automatically control the full microscope hardware and execute the desired measurement. However, it is unlikely that the user will have the skills or time to inspect

the resulting code, effectively precluding more detailed analysis, optimization, or further iteration. Alternatively, the AI could be directly integrated within a microscope control software, generating its output in a specific format that may or not be inspectable by non-experts users and that is in any case subject to specific assumptions about the operating environment, possible vendor lock-in, and all of the FAIR impediments this produces.

We think that this, too, is a language challenge. Successful collaboration with an AI agent, characterized by iterative discussion between the user and the agent, requires the use of shared language that can clearly capture the measurement particulars in a format that is understandable and clear to both. This is exactly the challenge already addressed by our work, and therefore the Imager DSL is a suitable intermediate to scaffold these interactions, aided by its clear and well-defined structure. In practical terms, the AI can be trained to recognize the Imager structure from example programs and a clear description of the components, after which it can automatically generate such programs. Because they are understandable to both parties, the resulting programs can be discussed and optimized while retaining their clarity and a full understanding of what they will do. Once judged sufficiently optimal, the generated program can then be directly submitted to the microscope for execution. While it is not directly integrated into the GUI at this point, we have readily confirmed this feasibility using commonly-available AI models.

In conclusion, we have presented Imager, a comprehensive microscope control software package focused on accessibility, flexibility, reproducibility, and FAIR data handling. To achieve this, Imager adopts an entirely new viewpoint that treats the measurement as a *language* challenge rather than as an engineering challenge. This viewpoint led us to propose a domain-specific language that embodies the knowledge and idioms present in fluorescence microscopy. This has a number of major, beneficial consequences. One of the most important of these is that creating well-defined measurement programs becomes straightforward even for less technically proficient users, since they can directly manipulate the program using convenient graphical operations. Furthermore, the DSL-embedded measurement program decouples the meaning (semantics) of the measurement program from the particulars of the microscope software and hardware implementation. This supports FAIR data usage by automatically including the full information specifying the measurement in a meaningful way, while remaining independent of the particulars of the software actually implementing the measurement. Overall, our approach mitigates or solves long-standing challenges in controlling complex experimental equipment.

We created an implementation consisting of a GUI front end, written in C#, and a microscope controller written in Haskell. This implementation has been used for over 10 years within the host research group, powering a wide variety of experiments. The source code of all of these components is freely available online. To our knowledge, this is the only example of a complicated measurement instrument being controlled in the Haskell programming language. While it is unconventional, our choice for Haskell has had an extremely positive impact on the reliability of the overall program and has also enabled complex refactors to be realized with confidence. Overall, we expect that many software projects, including real-time device control, can be made more reliable by switching to a functional programming language. The soundness of the overall design is made evident in the many applications that it has enabled within the hosting research group.

We also showed how our DSL-based approach can be extended towards ‘smart’ microscopy, delivering a novel architecture in which very strong flexibility is provided while still fitting with the overall vision of DSL-mediated instrument control, including directly exposing the overall control flow in a way that is intuitively understandable. Furthermore, our architecture enables the ‘smart’ programs to focus purely on the decision-making logic, removing the complexity associated with the hardware handling or measurement sequencing. The resulting programs and data retain their clarity and FAIR nature by explicitly incorporating the resulting programs within the DSL, as well as keeping a log of all ‘smart’ decisions directly within the overall measurement program.

Our language-driven approach also offers excellent prospects for the integration of AI-driven assistance and optimization. By providing a common language that is both clear to the user and to the AI, we enable the use of collaborative AI usage in a way that supports direct dialogue between both parties. Once judged sufficiently optimal, the generated programs can be directly submitted to

microscope for execution, retaining all of the FAIR and other advantages inherent to the approach.

More generally, our approach is suitable for any situation in which complex experimental equipment is used, including applications beyond fluorescence microscopy in other methodologies or settings. By providing the tools to reason and describe measurement programs in a platform neutral way, it offers the ability to ensure productive use by non-expert users, enables new experimental strategies, avoids vendor lock-in, dramatically simplifies FAIR data use, and supports innovative applications such as ‘smart’ imaging and AI-driven measurement optimization.

Data Availability

The full source code of the software described here is available at <https://github.com/ImagerMicroscopy>.

Acknowledgements

We thank Birthe Van den Berg and Wouter Lambrecht (KU Leuven) for assistance with the Haskell implementation. We thank Koen Martens (TU Delft) for inspirational discussions. We acknowledge support from the Research Foundation Flanders through grants G090819N and G010723N, the KU Leuven via C14/22/088, and the European Research Council via grant 714688 NanoCellActivity.

Author Contributions

PD designed the overall concept, wrote the control software and C++ plugins, and wrote the initial GUI based on Igor Pro. SA wrote the C# GUI and developed the smart microscopy implementation together with PD. TS provided input on the language design and Haskell implementation. GCC and RVDE acquired data.

References

- [1] Thijs Roebroek et al. “Reduced Fluorescent Protein Switching Fatigue by Binding-Induced Emissive State Stabilization”. In: *Int J Mol Sci* 18.9 (Sept. 2017), p. 2015. doi: 10.3390/ijms18092015.
- [2] Thijs Roebroek et al. “Simultaneous readout of multiple FRET pairs using photochromism”. In: *Nat Commun* 12.1 (Mar. 2021), p. 2005. doi: 10.1038/s41467-021-22043-0.
- [3] Hana Valenta et al. “Separation of spectrally overlapping fluorophores using intra-exposure excitation modulation”. In: *Biophys Rep (N Y)* 1.2 (Dec. 2021), p. 100026. doi: 10.1016/j.bpr.2021.100026.
- [4] Alison G Tebo et al. “Orthogonal fluorescent chemogenetic reporters for multicolor imaging”. In: *Nat Chem Biol* 17.1 (Jan. 2021), pp. 30–38. doi: 10.1038/s41589-020-0611-0.
- [5] Franziska Bierbuesse et al. “Absolute measurement of cellular activities using photochromic single-fluorophore biosensors and intermittent quantification”. In: *Nat Commun* 13.1 (Apr. 2022), p. 1850. doi: 10.1038/s41467-022-29508-w.
- [6] Anaïs C Bourges et al. “Quantitative determination of the full switching cycle of photochromic fluorescent proteins”. In: *Chem Commun (Camb)* 59.57 (July 2023), pp. 8810–8813. doi: 10.1039/d3cc01617j.
- [7] Vincent Van Deuren et al. “Simple generation of cleavable labels for multiplexed imaging”. In: *Chem Commun (Camb)* 60.97 (Dec. 2024), pp. 14403–14406. doi: 10.1039/d4cc01909a.
- [8] Robin Van den Eynde et al. “Simultaneous multicolor fluorescence imaging using PSF splitting”. In: *Nat Methods* 21.10 (Oct. 2024), pp. 1801–1805. doi: 10.1038/s41592-024-02383-7.

- [9] Hana Valenta et al. “Super-resolved Imaging of Molecular Interactions Using FRET-SOFI”. In: *Nano Lett* 25.47 (Nov. 2025), pp. 16690–16696. doi: 10.1021/acs.nanolett.5c04380.
- [10] Toon Van Thillo et al. “Reduced Artifacts in Nanoscale Ratiometric Imaging and Sensing via Point-Spread Function Matching”. In: *ACS Nano* 19.25 (July 2025), pp. 22755–22766. doi: 10.1021/acsnano.4c16338.
- [11] Xiong Chen et al. “Harnessing Methyltransferase-Guided Targeting for Sequence-Specific Proximity Labeling of DNA”. In: *Angew Chem Int Ed Engl* 65.15 (Apr. 2026), e20412. doi: 10.1002/anie.202520412.
- [12] Elke De Zitter et al. “Structure-Function Dataset Reveals Environment Effects within a Fluorescent Protein Model System”. In: *Angew Chem Int Ed Engl* 60.18 (Apr. 2021), pp. 10073–10081. doi: 10.1002/anie.202015201.